



Wechat:



LLM reason
Post training
RL theory
Agentic RL



CodeHacker

Automated Test Case Generation for Detecting Vulnerabilities in Competitive Programming Solutions

Xinxiang Yin^{2*} Jingwei Shi^{1*} Jing Huang^{3*} Shengyu Tao¹ Jinman Zhao^{4†}

¹ Shanghai University of Finance and Economics ² School of Software, Northwestern Polytechnical University
³ Meituan ⁴ University of Toronto * Equal contribution † Corresponding author



Oral presentation
@ ACL 2026



Motivation & task

Weak test suites accept incorrect programs. CodeHacker finds implementation-specific bugs by analyzing submissions, generating adversarial inputs, and refining them with execution feedback.

What is a successful hack?

A valid input must pass the verified reference solution and trigger a target failure: Wrong Answer, Runtime Error, Time Limit Exceeded, or Memory Limit Exceeded. Targets pass local tests but fail the official judge.

Two-phase method

I. Calibrate the judge

Probe validators for invalid inputs accepted or valid inputs rejected. Probe checkers for incorrect outputs accepted or valid outputs rejected. Patch both against the problem specification. Fewer than 5% of cases need expert intervention.

II. Generate & refine

Analyze vulnerabilities with C++ probes and Python calculations. Combine boundary stress tests, LLM-guided semantic tests, and anti-hash collisions. C++ generators create large inputs beyond output-token limits. Refine failed attempts for up to 5 turns using execution feedback. Obtain expected outputs from the verified reference solution.

Benchmark re-evaluation

Model	CC+	Ours	Δ pp
DeepSeek V3.2	83.3	81.6	-1.7
GPT-5-Mini	78.4	76.7	-1.7
Gemini-3 Flash	78.6	76.5	-2.1
DeepSeek V3.2*	61.9	59.7	-2.2
Qwen3-32B*	51.0	47.5	-3.5

Pass@1 (%): CodeContest+ vs. CodeContest++. Stronger tests expose accepted errors and lower scores by 1.7–3.5 pp. * Non-thinking mode.

CodeHacker framework

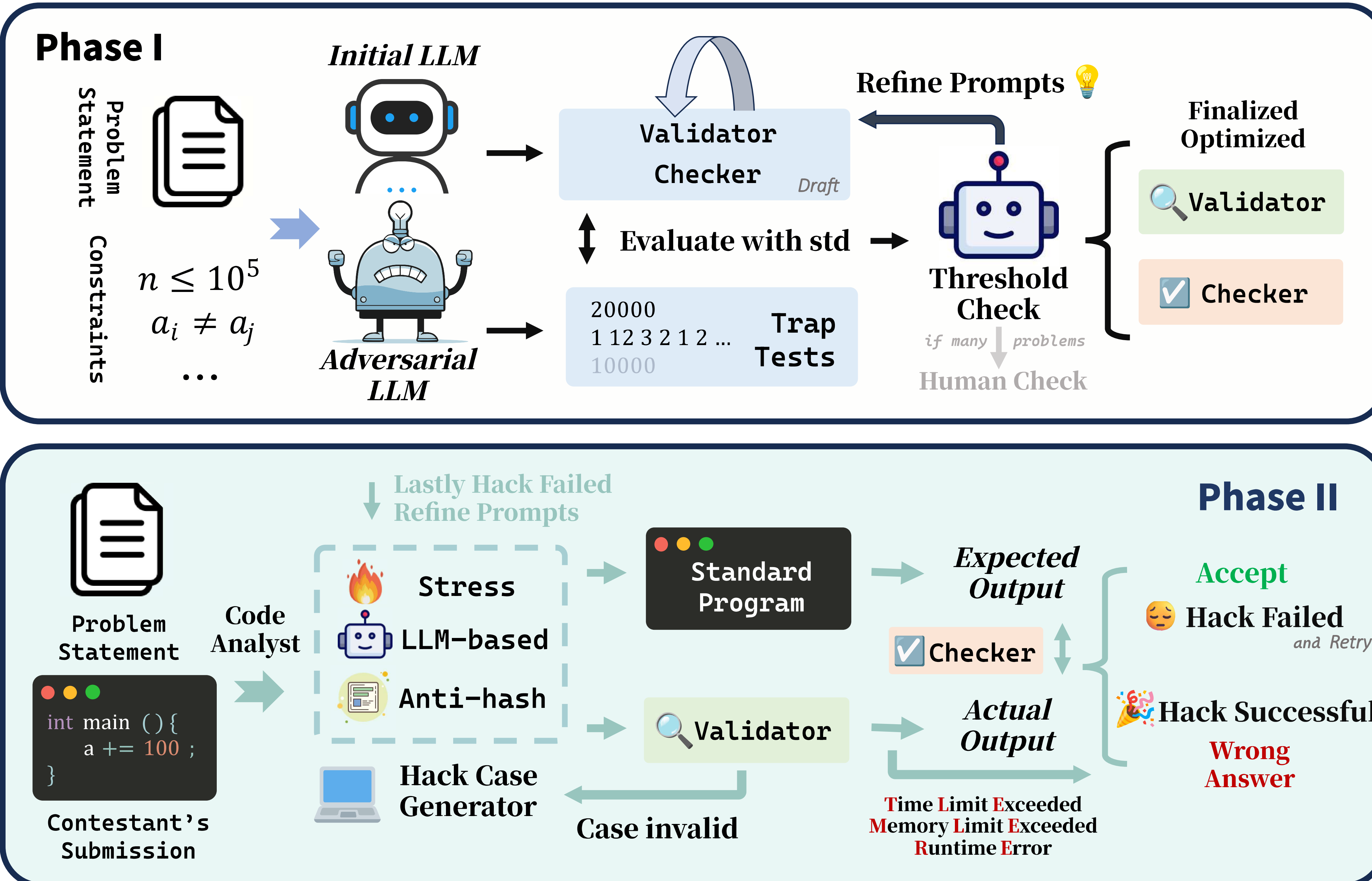


Figure 1. Phase I calibrates validators and checkers. Phase II combines code analysis, three complementary generators, and execution-guided refinement.

Test-suite quality on 2,000 problems

Dataset	VPR (%)	Trad. TPR	Trad. TNR	SPJ TPR	SPJ TNR
CodeContests	71.41	98.96	76.33	84.73	77.69
HardTests	97.32	98.33	79.25	86.56	75.37
TACO	81.84	96.46	83.70	82.67	85.48
CodeContest+	99.66	96.02	85.72	96.62	84.04
CodeContest++ (ours)	100.00	95.86	96.31	96.38	96.05

Metrics in %. Evaluation covers 1,000 traditional and 1,000 special-judge problems using calibrated validators and checkers.

TNR gain: +10.59 pp (traditional) and +12.01 pp (special judge)

VPR measures input validity. TNR measures rejection of incorrect submissions. CodeContest++ achieves 100% validity and improves TNR in both judging settings.

Hacking performance

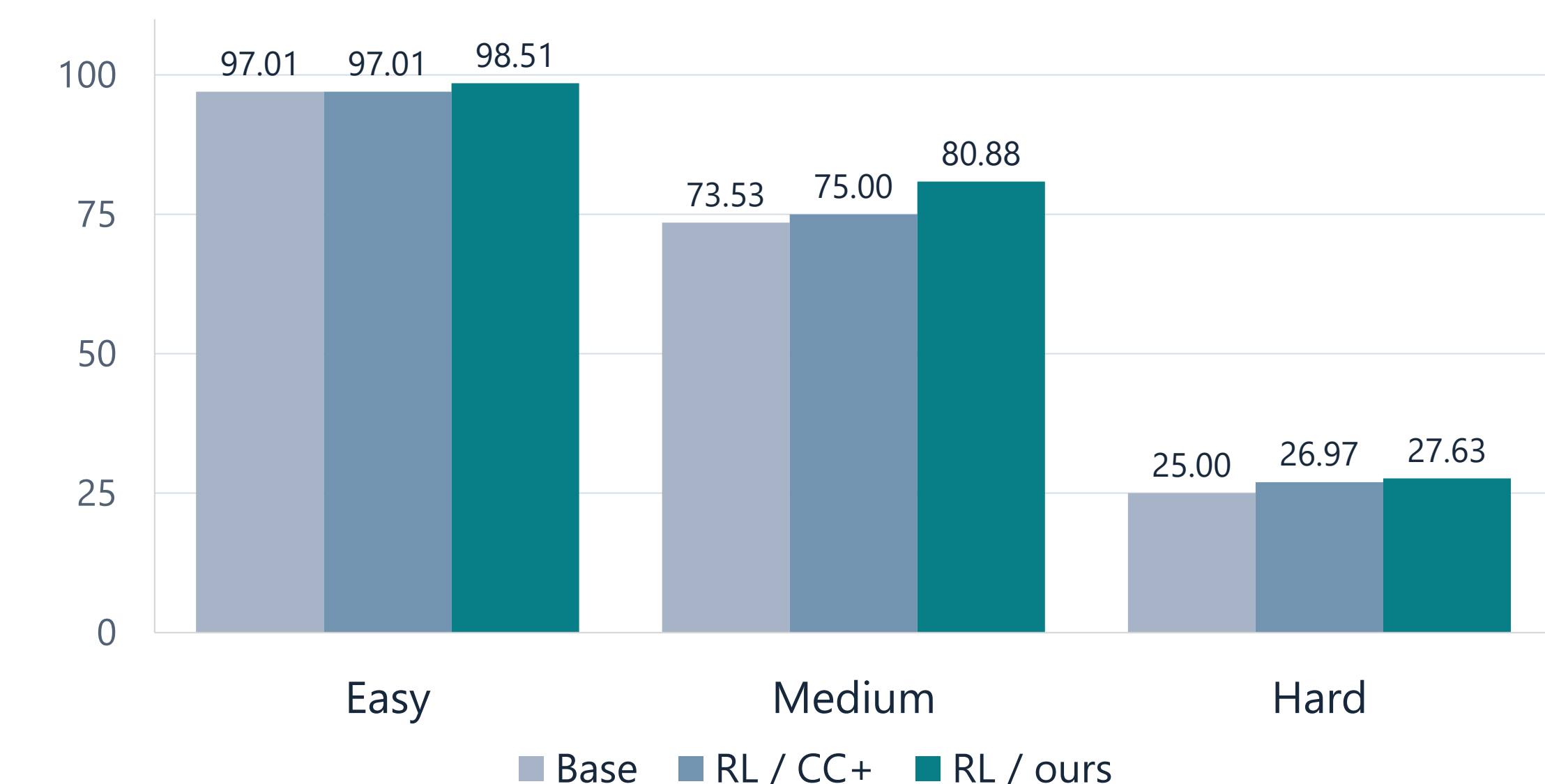
Backbone model	HSR %	Turns
DeepSeek V3.2	64.83	1.24
GPT-5-Mini	51.40	1.35
Qwen3-Next-80B-A3B	38.37	1.58
Gemini-3.0-Flash-Preview	35.65	1.62
Qwen3-32B	23.64	1.89
DeepSeek V3.2*	23.76	2.45
Qwen3-Next-80B-A3B*	20.06	2.68
Qwen3-32B*	10.00	2.91

HSR: hack success rate. Turns: mean turns for successful LLM-generated hacks (up to 5). * Non-thinking mode.

Ablation: GPT-5-Mini

Variant	HSR (%)
Full CodeHacker	51.40
Without Code Analyst	49.86
Without Stress Test	50.12
Without Anti-Hash	51.00
Without Refinement Loop	46.60

Adversarial training data



LiveCodeBench Pass@5 (%), 287 AtCoder problems. Qwen3-4B + DAPO: +5.88 pp on medium tasks vs. training with CodeContest+.

CodeHackerBench

~6,000 problem-submission pairs with adversarial tests. Experts audited every refined tool and 600 sampled hacks. All sampled hacks were valid and triggered target failures, supporting reliable evaluation and RL training.